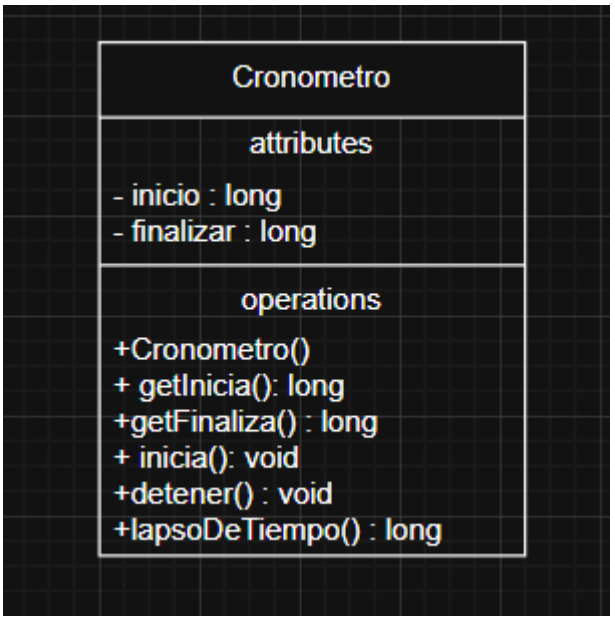
Laboratorio 1

NOMBRE:Raul Oscar Morales Flores NI: 9982323

EJERSICIOS

Cronómetro. Diseñé una clase llamada Cronometro. La clase contiene: a) Los atributos privados inicia y finaliza con métodos getter. b) Un constructor sin argumentos que inicializa inicia con la hora actual. c) Un método llamado inicia() que restablece inicia a la hora actual. d) Un método llamado detener() que establece finaliza a la hora actual. e) Un método llamado lapso De Tiempo() que retorna el tiempo transcurrido del cronómetro en milisegundos. Dibuje el diagrama UML de la clase e implem´entela. Escriba un programa de prueba que mida el tiempo de ejecución de la ordenación de 100.000 números mediante la ordenación por selección.

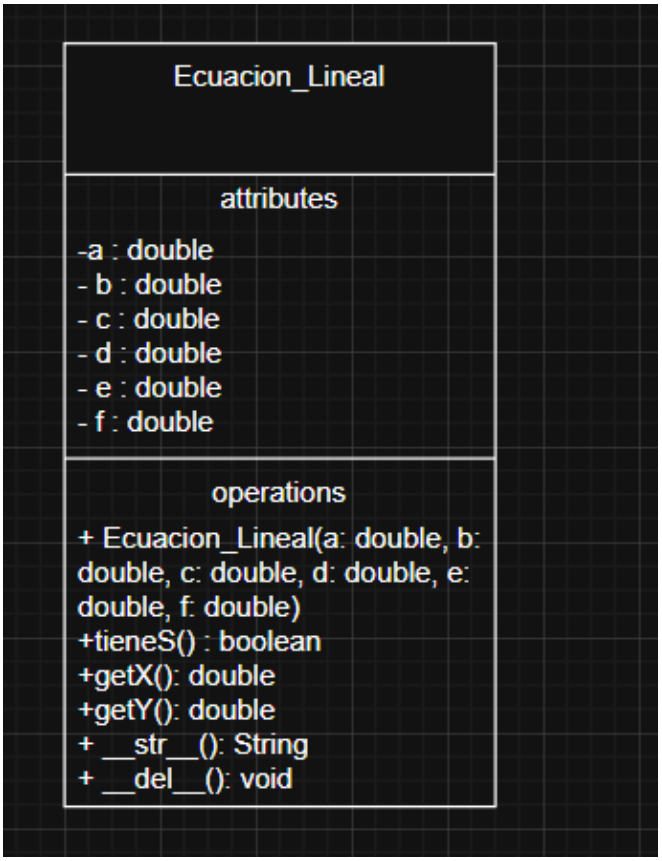



```
package appcronometro;
public class AppCronometro {
    public static int[] generarNumeros(int n) {
        int[] a = new int[n];
        for (int i = 0; i < n; i++) {
            a[i] = (int)(Math.random() * 1_000_000);
        }
        return a;
    }
    public static void seleccion(int[] a) {
        int n = a.length;
        for (int i = 0; i < n - 1; i++) {
            int min = i;
            for (int j = i + 1; j < n; j++) {
                if (a[j] < a[min]) min = j;
            }
            int tmp = a[i];
```

```
            a[i] = a[min];
            a[min] = tmp;
        }
    }

    public static void main(String[] args) {
        int[] datos = generarNumeros(100000);
        Cronometro c = new Cronometro();
        c.inicia();
        seleccion(datos);
        c.detener();

        System.out.println("Tiempo (ms): " + c.lapsoDeTiempo());
    }
}
```

```
package appcronometro;

public class Cronometro {
    private long inicia;
    private long finaliza;

    public Cronometro() {
        this.inicia = System.currentTimeMillis();
        this.finaliza = this.inicia;
    }

    public long getInicia() {
        return inicia;
    }
    public long getFinaliza() {
        return finaliza;
    }
    public void inicia() {
        this.inicia = System.currentTimeMillis();
    }

    public void detener() {
        this.finaliza = System.currentTimeMillis();
    }
    public long lapsoDeTiempo() {
        return finaliza - inicia;
    }}
```

2. Álgebra: ecuación lineal 2 x 2. Diseñe la clase Ecuación Lineal para un sistema de ecuaciones de 2 x 2.
ax +by =e
cx +dy =f
x = ed−bf
ad −bc y=af−ec
ad −bc
La clase contiene:
a) Los atributos privados a, b, c, d y f.
b) Un constructor para los argumentos para a, b, c, d y f.
c) Un método tiene Solución() que devuelve true si ad − bc no es cero.
d) Dos métodos getX() y getY() que retornan la solución de la ecuación.
Dibuje el diagrama UML de la clase e implemente . Escriba un programa de prueba
(Test) que solicite al usuario ingresar a, b, c, d, e y f y muestre el resultado. Si ad−bc es
0, indique que ”La ecuación no tiene solución”.
Ejemplo de entrada
Ingrese a, b, c, d, e, f: 9.0 4.0 3.0-5.0-6.0-21.0
Ingrese a, b, c, d, e, f: 1.0 2.0 2.0 4.0 4.0 5.0
Ejemplo de salida
x =-2.0, y = 3.0
La ecuación no tiene solución



```
class Ecuacion_Lineal:
    def __init__(self, a,b, c ,d,e,f):
        self.a = a
        self.b = b
        self.c = c
        self.d = d
        self.e = e
        self.f = f
    def tieneS(self):
        x = self.a * self.d - self.b * self.c
        return x !=0.0
    def getX(self):
        if (self.tieneS()):
            return (self.e * self.d - self.b * self.f) / (self.a *
self.d - self.b * self.c)
        else:
            return "la ecuacion no tiene solucion"
    def getY(self):
        if (self.tieneS()):
            return (self.a * self.f - self.e * self.c) / (self.a *
self.d - self.b * self.c)
        else:
            return "la ecuacion no tiene solucion"
    def __str__(self):
        return "{},{},{},{}".format(self.a, self.b, self.c, self.d)
t1 =  Ecuacion_Lineal(9.0,4.0,3.0,-5.0,-6.0,-21.0)
print(t1.tieneS())
print("X = ",t1.getX())
print("y = ",t1.getY())
t2 =  Ecuacion_Lineal(1.0,2.0,2.0,4.0,4.0,5.0)
print(t1.tieneS())
print("X = ",t2.getX())
print("y = ",t2.getY())
```

3. (*Álgebra: Ecuaciones Cuadráticas*) Diseñe la clase *EcuacionLineal* para la ecuación cuadrática $ax^2 + bx + c = 0$. La clase contiene:

   a) Los atributos privados a, b y c, que representan a los 3 coeficientes.

   b) Un constructor para los argumentos para a, b y c.

c) Un método **getDiscriminante()** que develve el discriminante $b^2 - 4ac$.

d) Dos métodos **getRaiz1()** y **getRaiz2()** que retornan las dos raíces de la ecuación.

$$r1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \qquad y \qquad r2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

Estos métodos solo son útiles si el discriminante es positivo. Si el discriminante es negativo, devuelva 0.

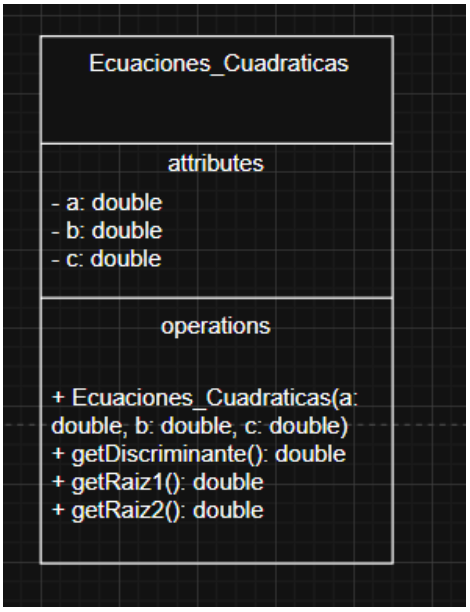
Dibuje el diagrama UML de la clase e impleméntela. Escriba un programa de prueba (Test) que solicite al usuario introducir los valores de a, b y c, y muestre el resultado según el discriminante. Si el discriminante es positivo, muestre las dos raíces. Si el discriminante es 0, muestre la única raíz. De lo contrario, muestre "La ecuación no tiene raíces reales".

Ejemplo de entrada

Ingrese a, b, c: `1.0 3 1`
Ingrese a, b, c: `1 2.0 1`
Ingrese a, b, c: `1 2 3`

Ejemplo de salida

```
La ecuación tiene dos raíces -0.381966 y -2.61803
La ecuación tiene una raíz -1
La ecuación no tiene raíces reales
```




```python
import math

class Ecuaciones_Cuadraticas:

    def __init__(self, a, b, c):
        self.__a = a
        self.__b = b
        self.__c = c
    def getDiscriminante(self):
        return self.__b**2 - 4*self.__a*self.__c
    def getRaiz1(self):
```

```
        d = self.getDiscriminante()
        if d < 0:
            return 0
        return (-self.__b + math.sqrt(d)) / (2*self.__a)
    def getRaiz2(self):
        d = self.getDiscriminante()
        if d < 0:
            return 0
        return (-self.__b - math.sqrt(d)) / (2*self.__a)
for i in range(3):
    a, b, c = map(float, input("Ingrese a, b, c: ").split())
    ecuacion = Ecuaciones_Cuadraticas(a, b, c)
    d = ecuacion.getDiscriminante()
    if d > 0:
        print(f"La ecuación tiene dos raíces {ecuacion.getRaiz1():.6f}
y {ecuacion.getRaiz2():.5f}")
    elif d == 0:
        print(f"La ecuación tiene una raíz {ecuacion.getRaiz1():.0f}")
    else:
        print("La ecuación no tiene raíces reales")
```

4. *Estadísticas: calcular el promedio y la desviación estándar.* En las aplicaciones comerciales, a menudo se le pide que calcule el promedio y la desviación estándar de los datos. La desviación estándar es una estadística que le indica cuán estrechamente se agrupan todos los diversos datos alrededor de la media en un conjunto de datos. Por ejemplo, ¿cuál es la edad promedio de los estudiantes en una clase? ¿Qué tan cerca están las edades? Si todos los estudiantes tienen la misma edad, la desviación es 0. Utilice la siguiente fórmula:

$$promedio = \frac{\sum_{i=1}^{n} x_i}{n} \qquad desviacion = \sqrt{\frac{\sum_{i=1}^{n}(x_i - promedio)^2}{n-1}}$$

Su programa debe contener los siguientes métodos:

- promedio() obtiene el promedio de los valores en punto flotante
- desviacion() devuelve la desviación estándar de los valores en punto flotante.

Escriba un programa que solicite al usuario ingresar 10 números, muestre el promedio y las desviaciones estándar de estos números.

Ejemplo de entrada

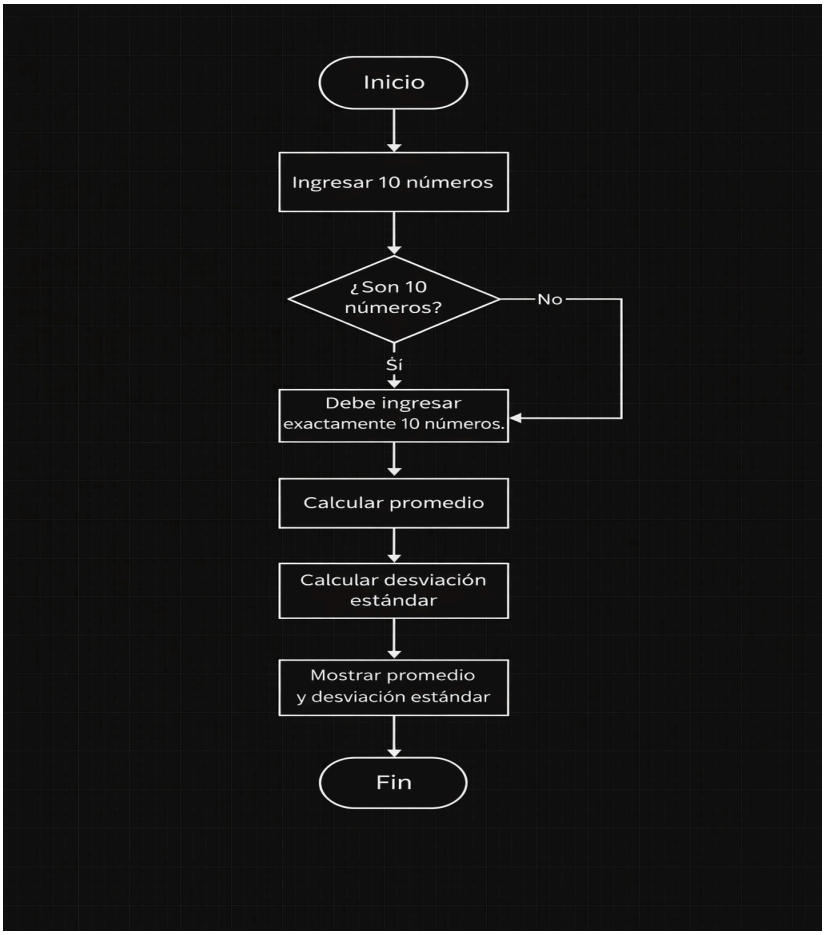
```
1.9 2.5 3.7 2 1 6 3 4 5 2
```

Ejemplo de salida

```
El promedio es 3.11
La desviación estandard es 1.55738
```

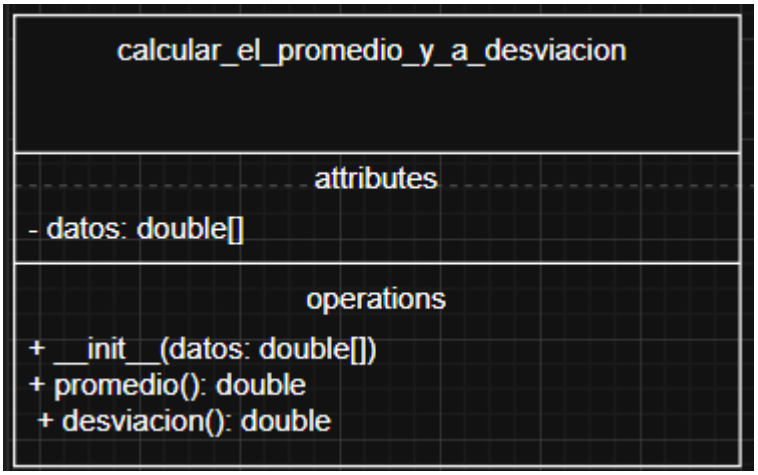
Obtenga la solución en la Programación Modular-Estructura y luego en Programación Orientada a Objetos, para ello dibuje el diagrama UML para la clase Estadistica y luego implemente la clase. Agregue de forma escrita las ventajas de utilizar la P.O.O.

Modular:




```python
import math
def promedio(datos):
    return sum(datos) / len(datos)
def desviacion(datos):
    n = len(datos)
    prom = promedio(datos)
    suma = 0
    for x in datos:
        suma += (x - prom) ** 2
    return math.sqrt(suma / (n - 1))
datos = list(map(float, input("Ingrese 10 números separados por
espacio: ").split()))
if len(datos) != 10:
    print("Debe ingresar exactamente 10 números.")
else:
    print(f"El promedio es {promedio(datos):.2f}")
    print(f"La desviación estándar es {desviacion(datos):.5f}")
```

POO:




```
import math
class calcular_el_promedio_y_a_desviacion:
    def __init__(self, datos):
        self.datos = datos
    def promedio(self):
        return sum(self.datos) / len(self.datos)
    def desviacion(self):
        n = len(self.datos)
        prom = self.promedio()
        suma = 0
        for x in self.datos:
            suma += (x - prom) ** 2
        return math.sqrt(suma / (n - 1))
datos = list(map(float, input("Ingrese 10 números separados por
espacio: ").split()))

if len(datos) != 10:
    print("Debe ingresar exactamente 10 números.")
else:
    est = calcular_el_promedio_y_a_desviacion(datos)
    print(f"El promedio es {est.promedio():.2f}")
    print(f"La desviación estándar es {est.desviacion():.5f}")
```

Agregue de forma escrita las ventajas de utilizar la P.O.O.

Ventajas de utilizar la Programación Orientada a Objetos (P.O.O.):

1. **Modularidad**:
    - La programación orientada a objetos permite organizar el código en módulos llamados **clases**. Cada clase es responsable de una parte del sistema. En este caso, la clase `Estadistica` encapsula las

funcionalidades relacionadas con los cálculos del promedio y la desviación estándar.

2. **Reutilización del código**:
    - Una vez definida la clase `Estadistica`, puedes reutilizarla múltiples veces sin necesidad de reescribir código. Puedes crear instancias de la clase para diferentes conjuntos de datos y obtener los resultados sin alterar la estructura básica.
3. **Encapsulamiento**:
    - La P.O.O. permite ocultar la implementación interna (como los cálculos del promedio y la desviación estándar) y proporciona métodos públicos para interactuar con los datos. Esto significa que el usuario no necesita saber cómo se calcula el promedio o la desviación estándar, solo tiene que usar los métodos proporcionados.
4. **Facilita la extensión del código**:
    - Si en el futuro se necesitan nuevas funcionalidades, como agregar más métricas estadísticas (mediana, moda, etc.), se puede agregar fácilmente sin alterar el código existente. Solo tienes que agregar un nuevo método a la clase `Estadistica`.
5. **Mantenimiento**:
    - La P.O.O. facilita el mantenimiento del código. Si se encuentra un error en un método, solo necesitas modificar esa parte de la clase sin afectar al resto del programa.
6. **Abstracción**:
    - La programación orientada a objetos permite abstraer detalles complejos de la implementación. El usuario solo interactúa con la clase a través de sus métodos públicos, sin necesidad de conocer los detalles internos del cálculo.

Conclusión:

- **Programación Modular**: Es más sencilla para problemas pequeños, pero puede volverse más difícil de mantener y ampliar a medida que el sistema

crece.

- **Programación Orientada a Objetos**: Ofrece mayor organización, modularidad y flexibilidad para proyectos más grandes, haciendo que el código sea más fácil de mantener y extender.